If only the world worked this way! Unfortunately, not only systems (applications, computers, networks etc.) do fail, but there is also a good chance that, for an e-commerce web application like the one we described, some crucial pieces are not even under our control. That is, we are likely dealing with a distributed application, that includes different services, running on different machines (possibly located far away from each other) and communicating through remote calls over a network. Figure 11.5 builds upon the simplified architecture in figure 11.3, depicting a more realistic situation, where the retailers' servers live on their separate machines, and thus in a different addressing space, only accessible through remote calls (over HTTP, or any other communication protocol, like *IPFS*[8]).

As you can imagine, this changes the rules of the game: while we can have a tight control on what runs in our machine (virtual or physical), once we introduce remote calls, we also introduce additional failure points and, additionally, we need to deal with latency. For instance, if we synchronously call a method in our application, we know it can fail, hopefully we also know why it would fail, and we know it can take some time to compute (or, possibly, even loop forever): but we are also sure that the method was called and started doing its job.